

COMPUTER VISION – ANNEXURE A

EXPERIMENTS

Name: V Sunil Kumar

Register No.: 192425292

EXPERIMENT 1: Feature Descriptor Comparison (SIFT vs Harris)

Aim: To compare Harris Corner Detection and SIFT feature extraction on the same image and study their feature quality, repeatability and usefulness for image matching.

Tools Used: Python 3, OpenCV (cv2), NumPy, Matplotlib (optional)

Theory / Concept: Harris mainly identifies strong corner locations, while SIFT gives keypoints with descriptors that can be used for matching. In this experiment, SIFT detected more detailed local features and is more suitable when the image may change in scale or rotation.

Procedure:

1. Read the input image and convert it to grayscale.
2. Apply Harris corner detection and mark the detected corners.
3. Create a SIFT detector and find keypoints and descriptors.
4. Draw the SIFT keypoints on the image.
5. Save both results and compare the detected features.

Python Code:

```
import cv2

img = cv2.imread("input1.png")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

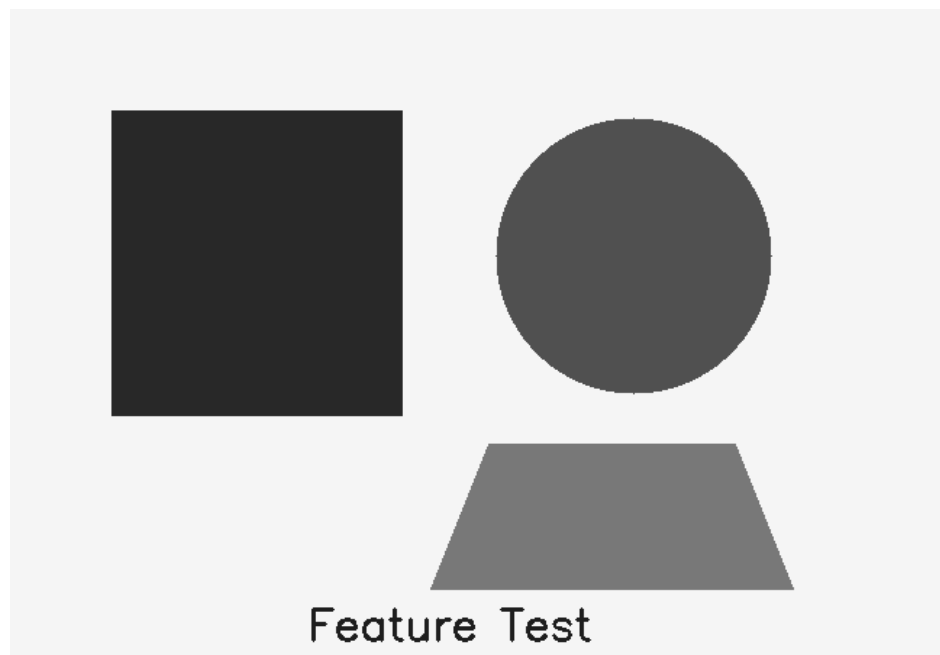
# Harris corner detection
gray32 = cv2.cornerHarris(gray.astype("float32"), 2, 3, 0.04)
harris = img.copy()
harris[gray32 > 0.01 * gray32.max()] = [0, 0, 255]

# SIFT feature extraction
```

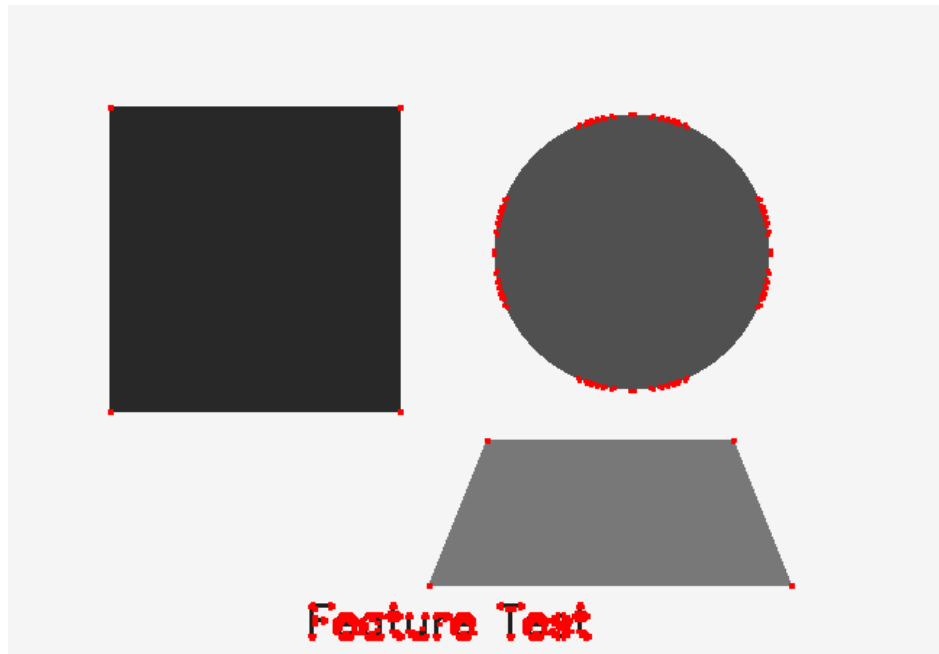
```
sift = cv2.SIFT_create()
keypoints, descriptors = sift.detectAndCompute(gray, None)
sift_img = cv2.drawKeypoints(
    img, keypoints, None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)
```

```
cv2.imwrite("harris_output.png", harris)
cv2.imwrite("sift_output.png", sift_img)
print("SIFT keypoints:", len(keypoints))
```

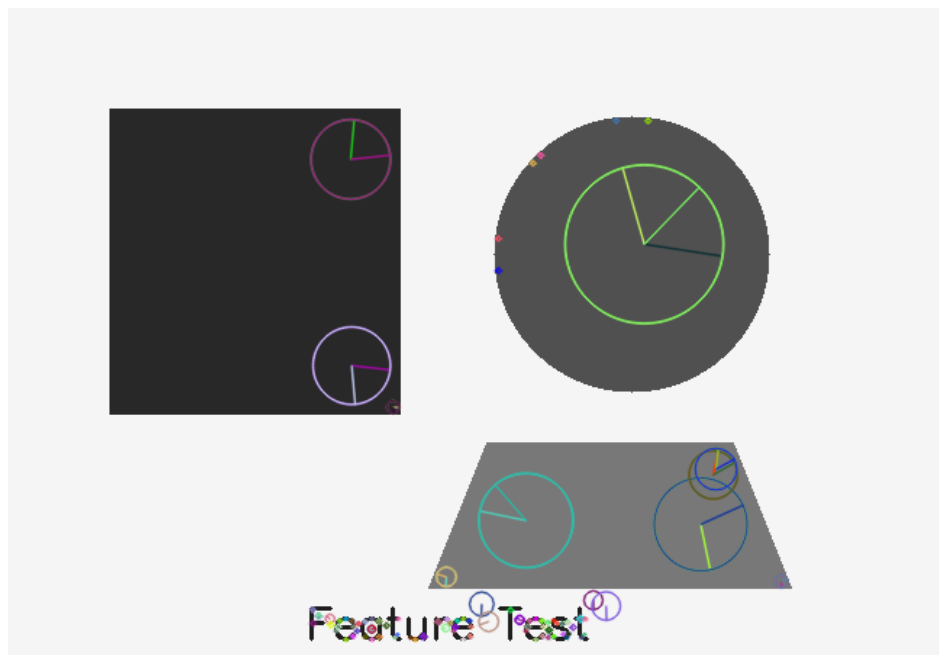
Input Image and Output Images:



Input image



Harris detected corners



Result: The result was obtained successfully. Harris highlighted corner points, while SIFT produced scale-aware keypoints. SIFT is more useful for image matching because it also provides descriptors. Number of SIFT keypoints detected: 128.

EXPERIMENT 2: Image Enhancement for Feature Detection

Aim: To study how image enhancement methods such as histogram equalization and contrast enhancement affect feature visibility and feature detection.

Tools Used: Python 3, OpenCV, NumPy, Jupyter Notebook / VS Code, sample image.

Theory / Concept: The enhanced image showed clearer boundaries and local details. Histogram equalization improves global contrast, while CLAHE works better when illumination is uneven. Better contrast can make useful features easier to detect.

Procedure:

1. Read the image and convert it into grayscale.
2. Apply histogram equalization to improve global contrast.
3. Apply CLAHE contrast enhancement to improve local contrast.
4. Run SIFT feature detection on the enhanced image.
5. Compare the original and enhanced feature visibility.

Python Code:

```
import cv2

img = cv2.imread("input2.png")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

equalized = cv2.equalizeHist(gray)

clahe = cv2.createCLAHE(
    clipLimit=2.0, tileGridSize=(8, 8)
)
enhanced = clahe.apply(gray)

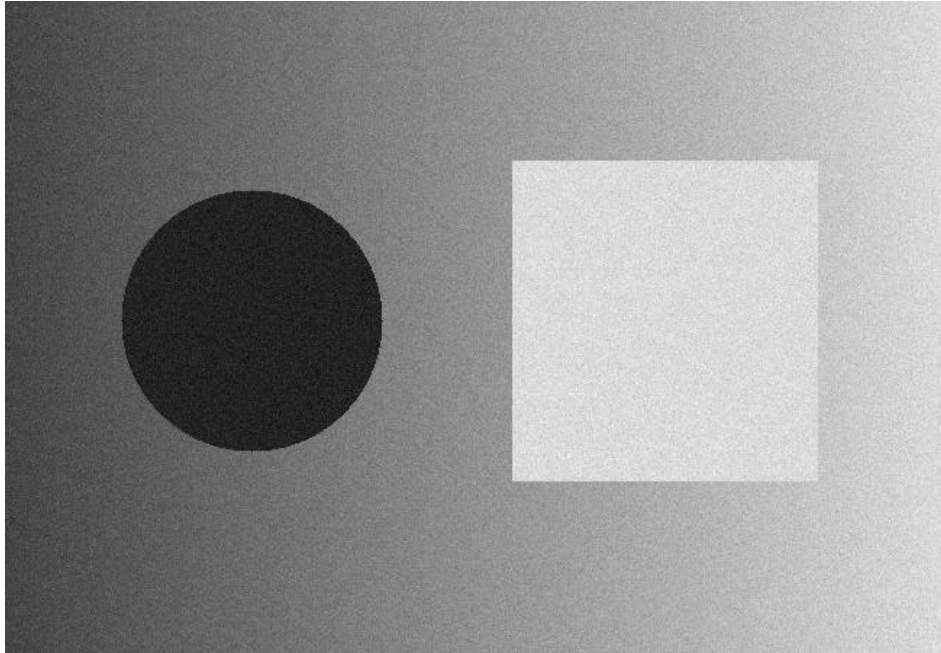
sift = cv2.SIFT_create()
kp, des = sift.detectAndCompute(enhanced, None)

result = cv2.drawKeypoints(
    enhanced, kp, None,
```

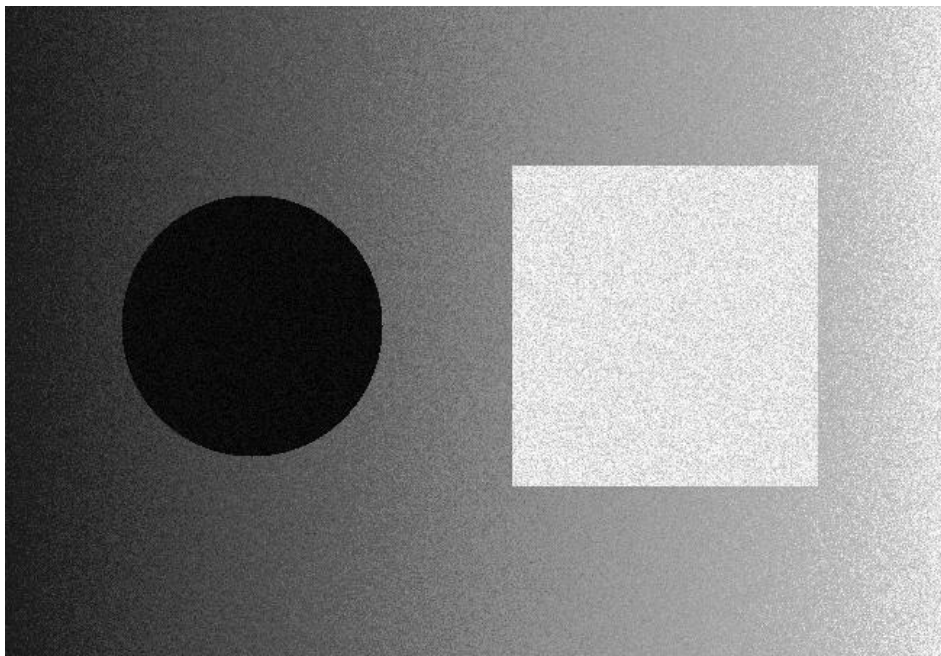
```
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS  
)
```

```
cv2.imwrite("equalized.png", equalized)  
cv2.imwrite("enhanced_keypoints.png", result)  
print("Detected features:", len(kp))
```

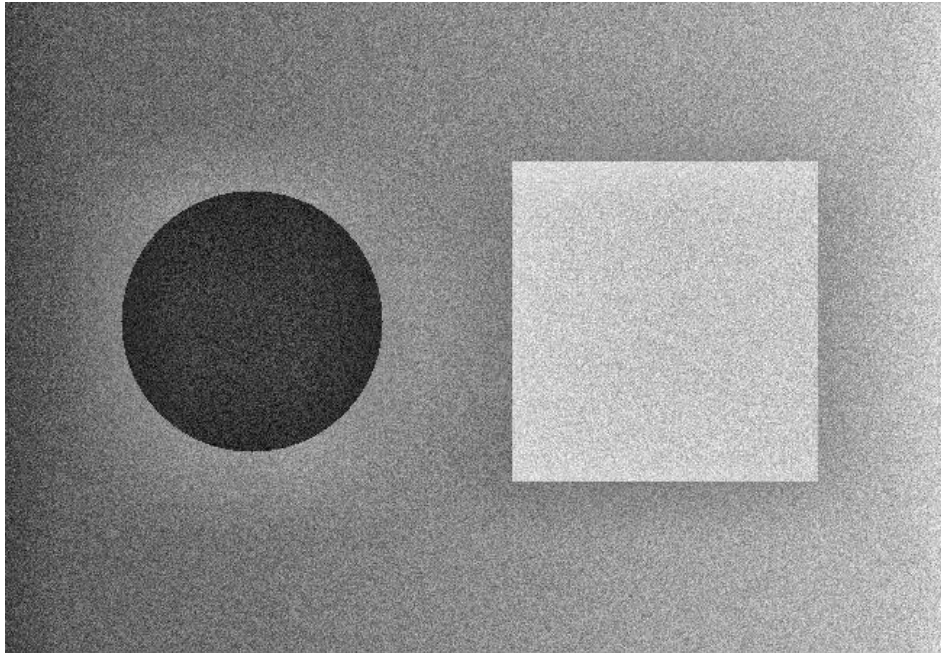
Input Image and Output Images:



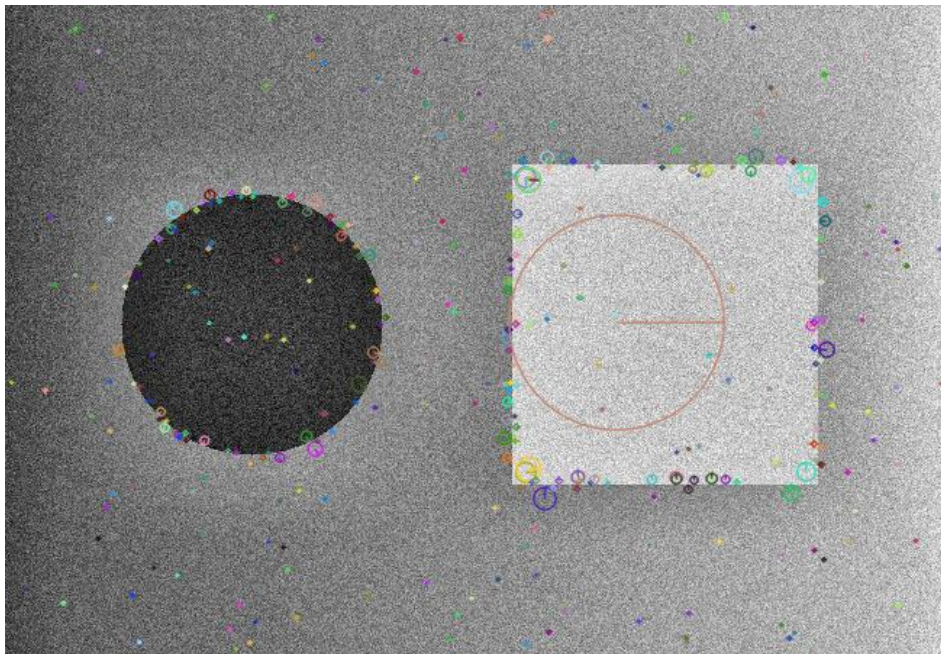
Input image



Histogram equalization



CLAHE enhancement



Features after enhancement

Result: Image enhancement improved the visibility of several local details. SIFT detected 436 features after CLAHE in the generated test image.

EXPERIMENT 3: Multi-Scale Feature Detection Analysis

Aim: To examine how changing the image scale affects feature detection, feature stability and computational requirements.

Tools Used: Python 3, OpenCV, NumPy, Jupyter Notebook / VS Code.

Theory / Concept: Scale changes affect the number and location of detected features. At smaller resolutions, some fine details disappear, so fewer useful features may be detected. Smaller images need less computation, but very small images can reduce detection accuracy.

Procedure:

1. Load the same input image.
2. Create resized versions at 100%, 75%, 50% and 25%.
3. Convert each resized image to grayscale.
4. Run SIFT on every scale.
5. Record the number of detected keypoints and compare the outputs.

Python Code:

```
import cv2

img = cv2.imread("input3.png")
sift = cv2.SIFT_create()

for scale in [1.0, 0.75, 0.5, 0.25]:
    resized = cv2.resize(img, None, fx=scale, fy=scale)
    gray = cv2.cvtColor(resized, cv2.COLOR_BGR2GRAY)

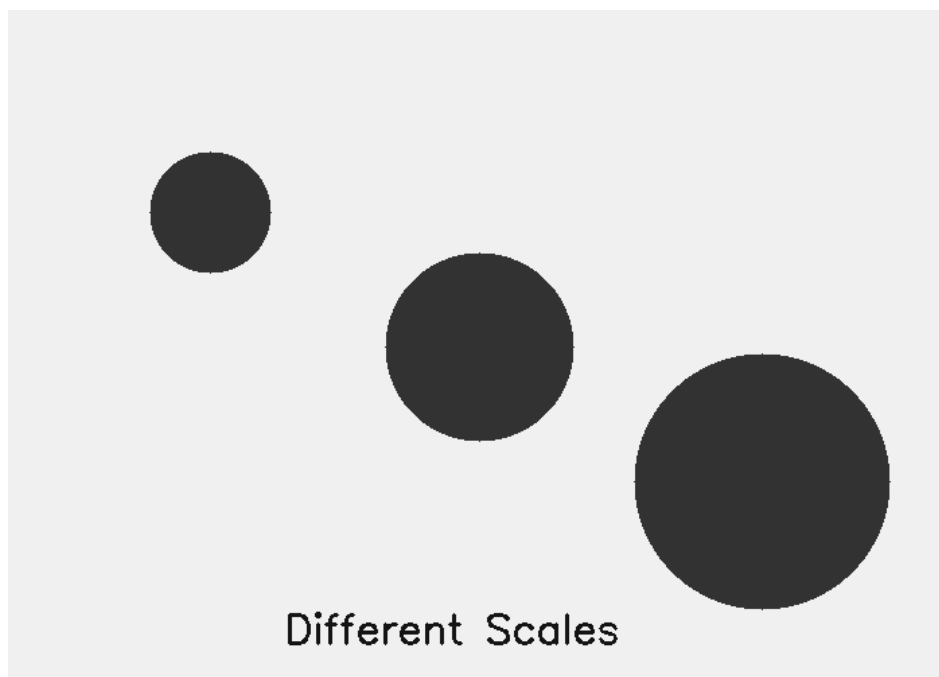
    keypoints, descriptors = sift.detectAndCompute(gray, None)

    output = cv2.drawKeypoints(
        resized, keypoints, None,
        flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
    )
```

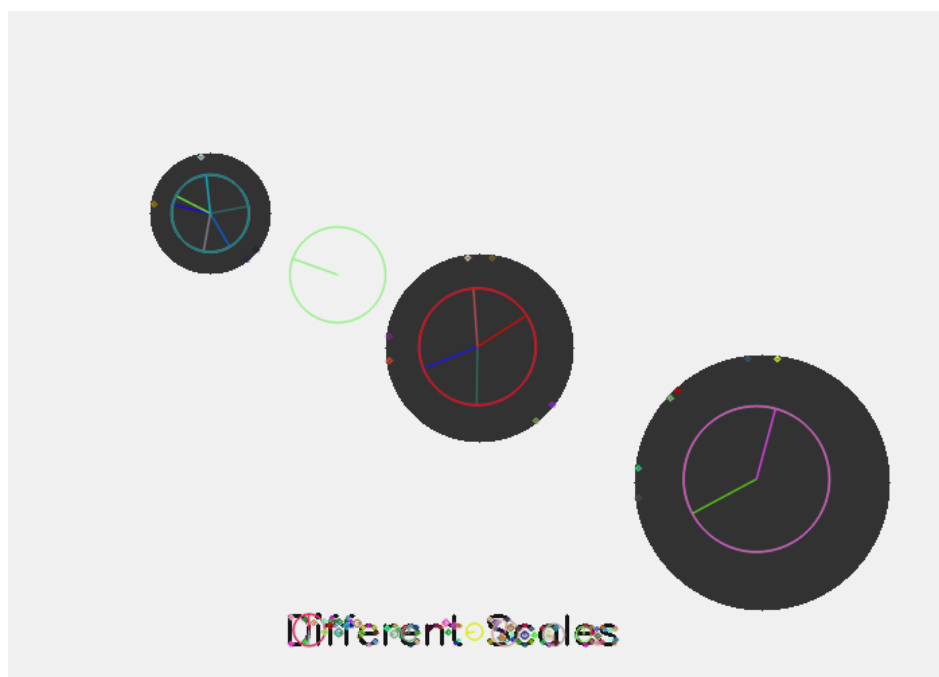


```
print("Scale:", scale,  
      "Keypoints:", len(keypoints))
```

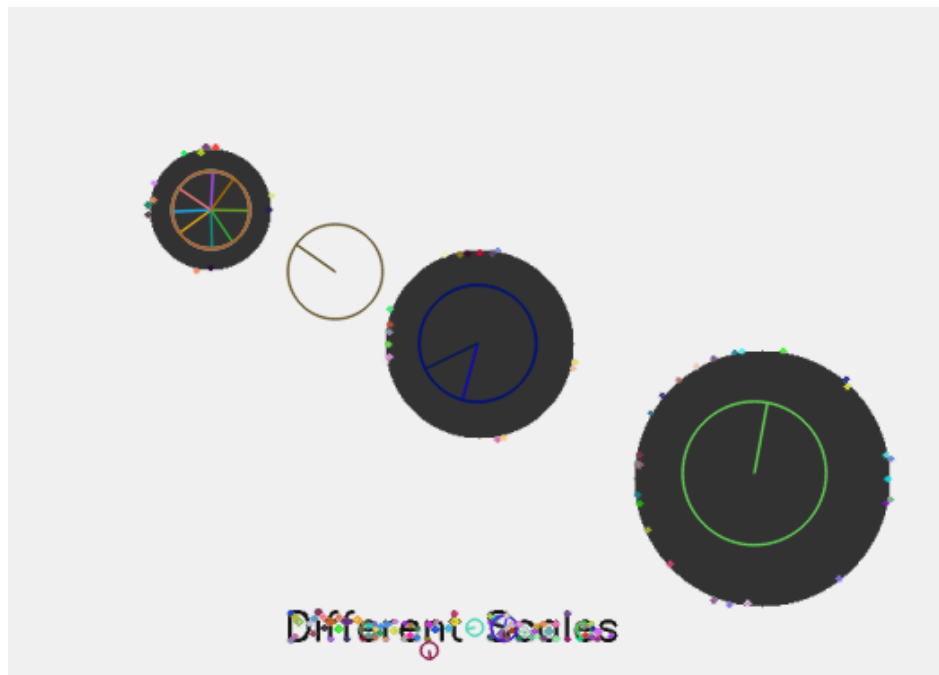
Input Image and Output Images:



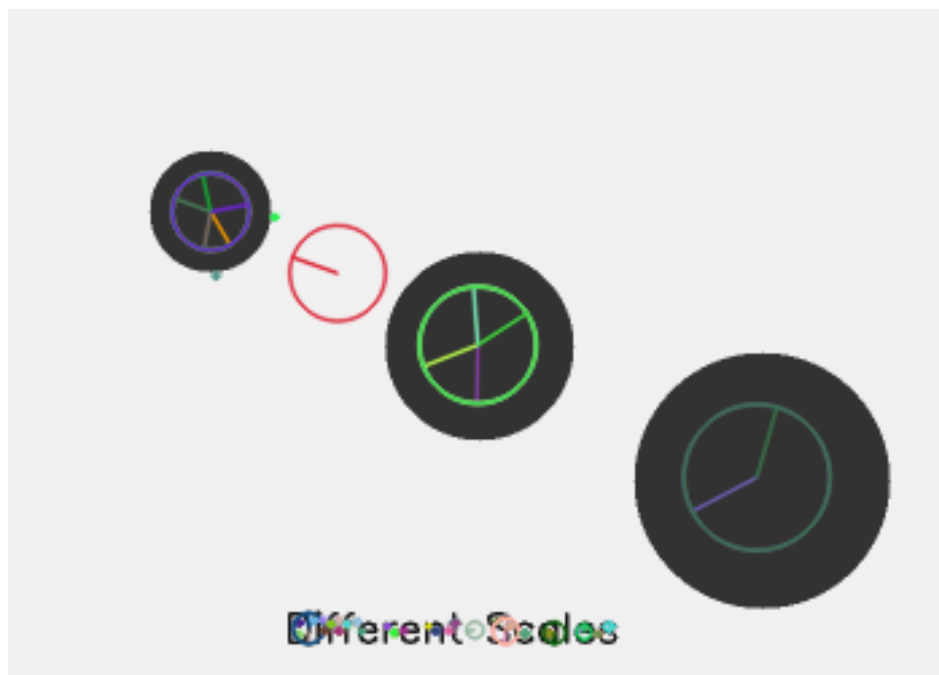
Input image



100% scale



75% scale



50% scale



25% scale

Result: Feature counts in the four generated scales were [173, 220, 69, 14]. The experiment shows that reducing resolution generally reduces fine feature information and also reduces processing work.

EXPERIMENT 4: Comparative Analysis of Image Filtering Techniques

Aim: To compare Mean, Gaussian and Median filters for noise reduction while trying to preserve useful image boundaries and features.

Tools Used: Python 3, OpenCV, NumPy, Jupyter Notebook / VS Code.

Theory / Concept: Mean filtering smooths the image but can blur edges. Gaussian filtering gives smoother and more natural results. Median filtering is especially useful for salt-and-pepper noise and usually preserves edges better than the mean filter.

Procedure:

1. Read the noisy input image.
2. Apply a 5×5 Mean filter.
3. Apply a 5×5 Gaussian filter.
4. Apply a 5×5 Median filter.
5. Visually compare smoothness, noise removal and edge preservation.

Python Code:

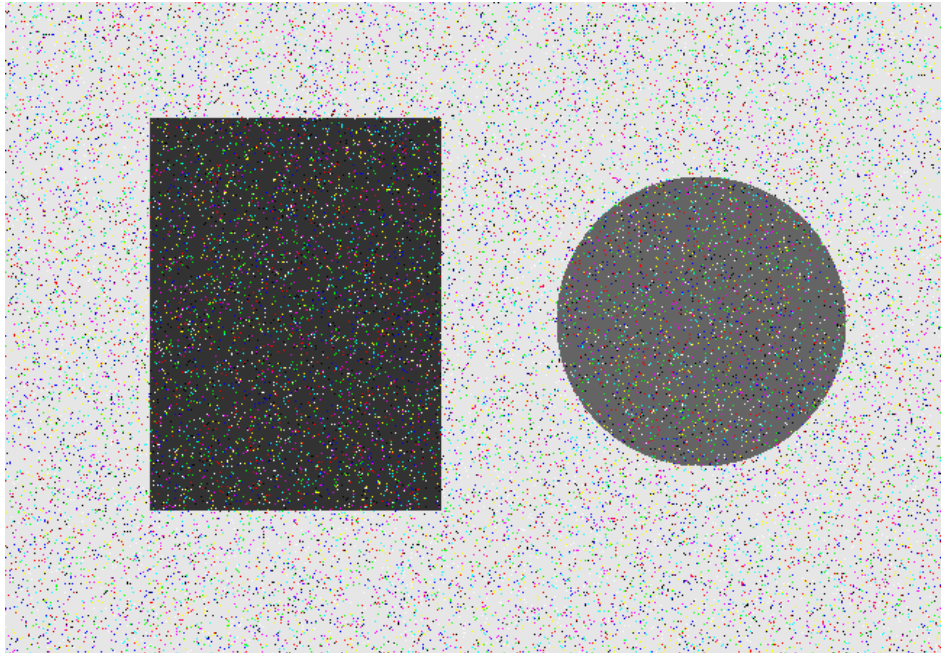
```
import cv2

img = cv2.imread("input4.png")

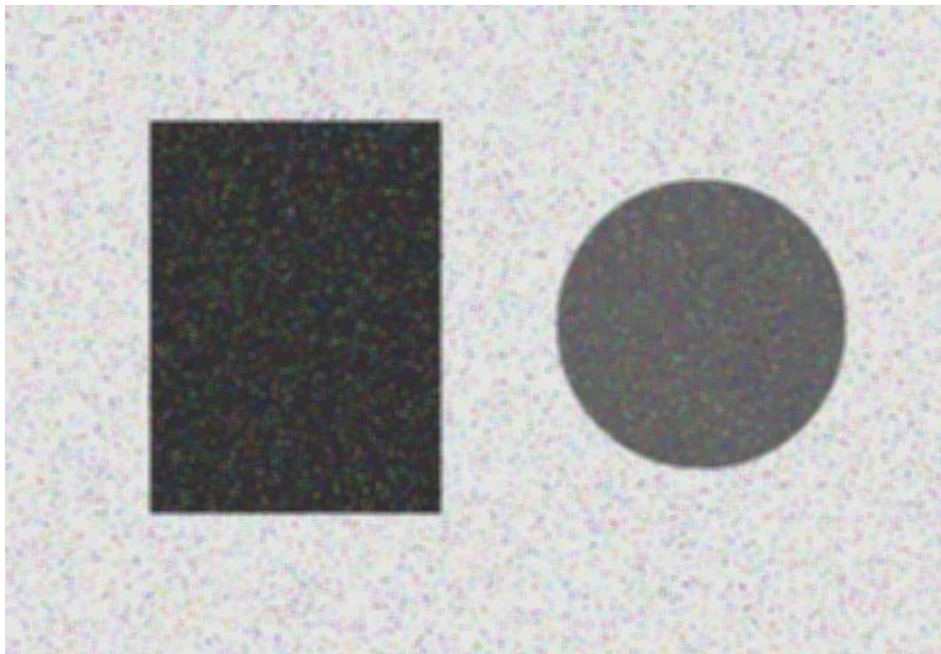
mean_filter = cv2.blur(img, (5, 5))
gaussian_filter = cv2.GaussianBlur(img, (5, 5), 0)
median_filter = cv2.medianBlur(img, 5)

cv2.imwrite("mean.png", mean_filter)
cv2.imwrite("gaussian.png", gaussian_filter)
cv2.imwrite("median.png", median_filter)
```

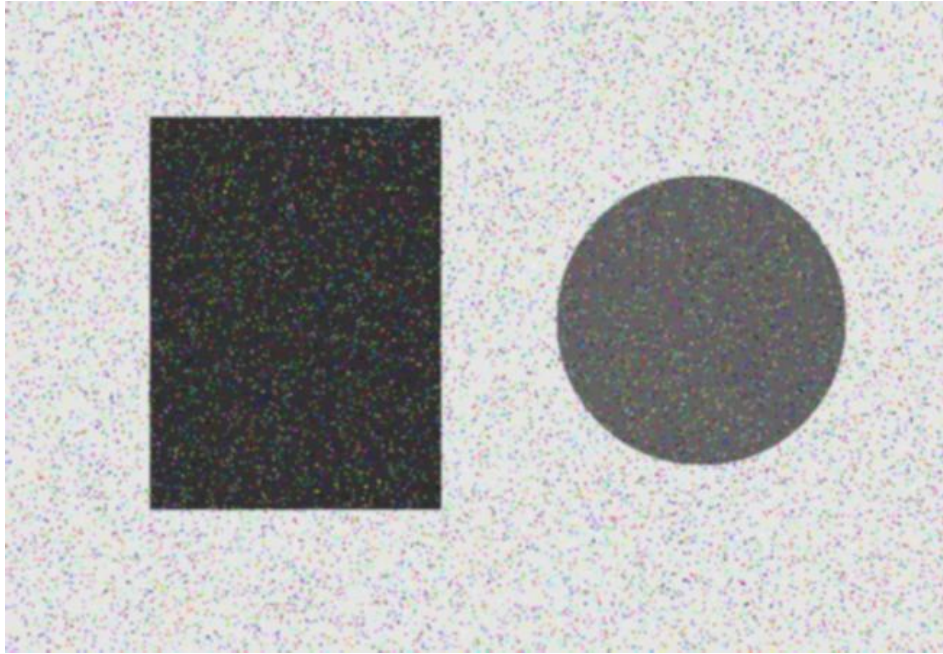
Input Image and Output Images:



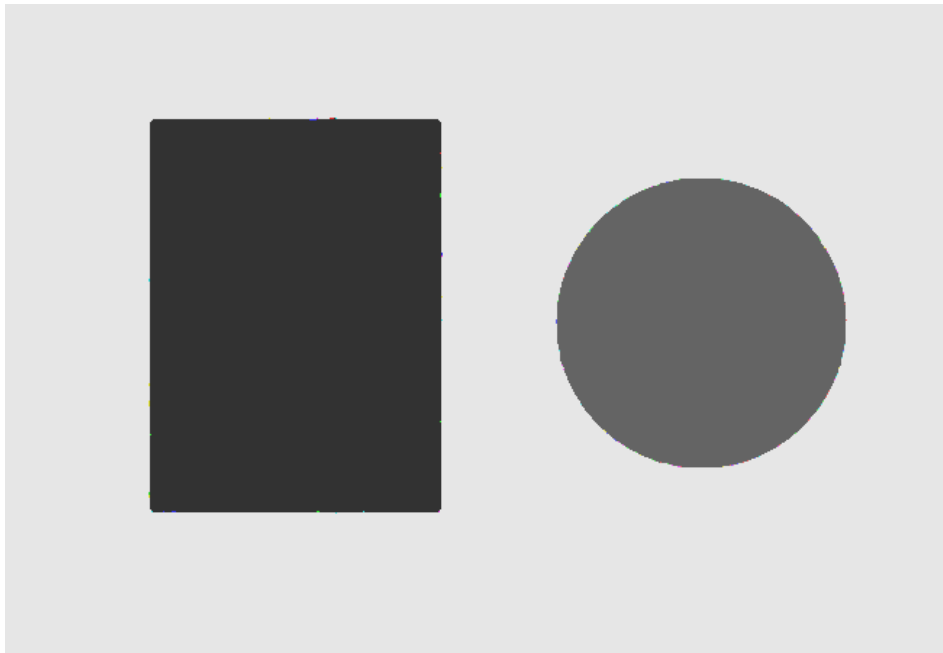
Noisy input image



Mean filtering



Gaussian filtering



Median filtering

Result: The median filter gave the cleanest result for the impulse-noise pattern in the test image. Gaussian filtering also reduced noise with moderate smoothing, while the mean filter caused more edge blurring.

EXPERIMENT 5: Complete Object Detection Workflow

Aim: To implement a simple object detection pipeline using preprocessing, edge detection, feature/shape extraction and contour-based shape detection.

Tools Used: Python 3, OpenCV, NumPy, Jupyter Notebook / VS Code.

Theory / Concept: The complete pipeline converts a raw image into a form where object boundaries can be detected. Preprocessing reduces noise, Canny finds edges, and contours provide the shape information needed for simple object detection.

Procedure:

- 1 Read the input image.
2. Convert the image to grayscale.
3. Use Gaussian filtering to reduce small noise.
4. Apply Canny edge detection.
5. Find external contours from the edge image.
6. Approximate contours and draw the detected shapes.
7. Compare the final output with the original image.

Python Code:

```
import cv2

img = cv2.imread("input5.png")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

blur = cv2.GaussianBlur(gray, (5, 5), 0)
edges = cv2.Canny(blur, 50, 150)

contours, _ = cv2.findContours(
    edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
)

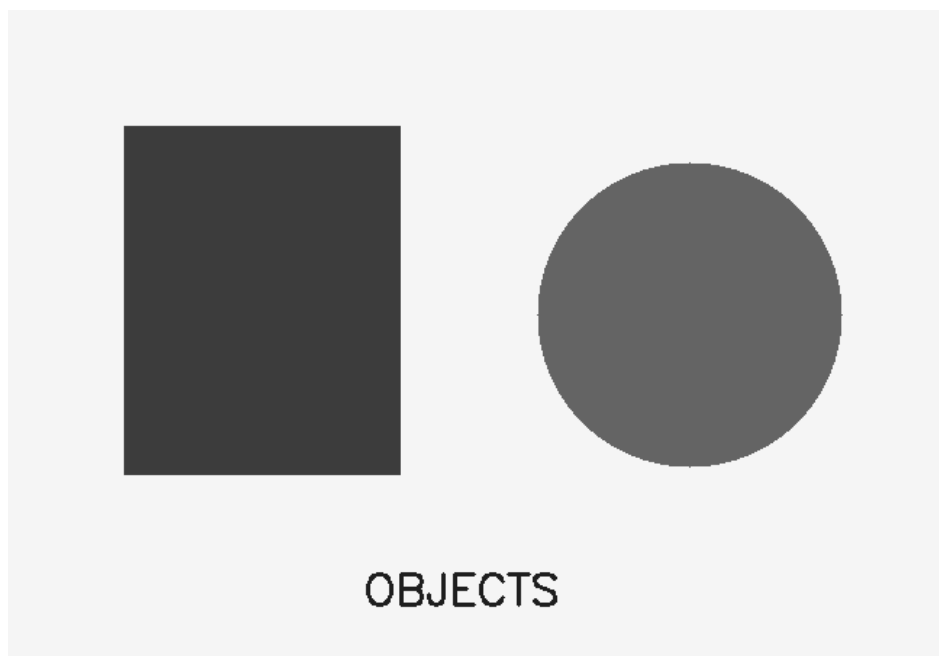
result = img.copy()

for contour in contours:
```

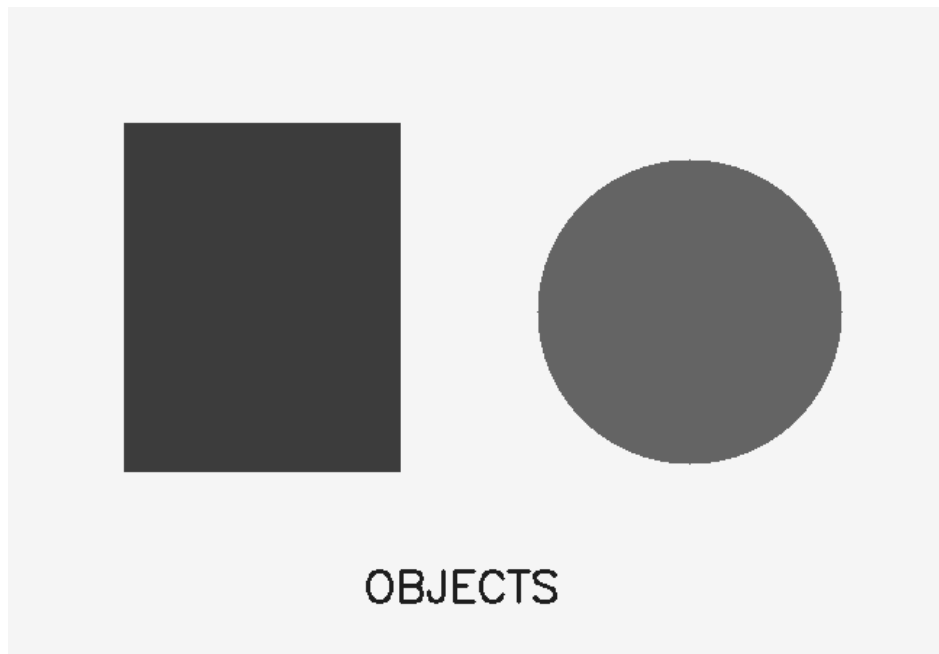
```
if cv2.contourArea(contour) > 500:  
    perimeter = cv2.arcLength(contour, True)  
    approx = cv2.approxPolyDP(  
        contour, 0.02 * perimeter, True  
    )  
    cv2.drawContours(result, [approx], -1, (0, 0, 255), 3)
```

```
cv2.imwrite("edges.png", edges)  
cv2.imwrite("detected_shapes.png", result)
```

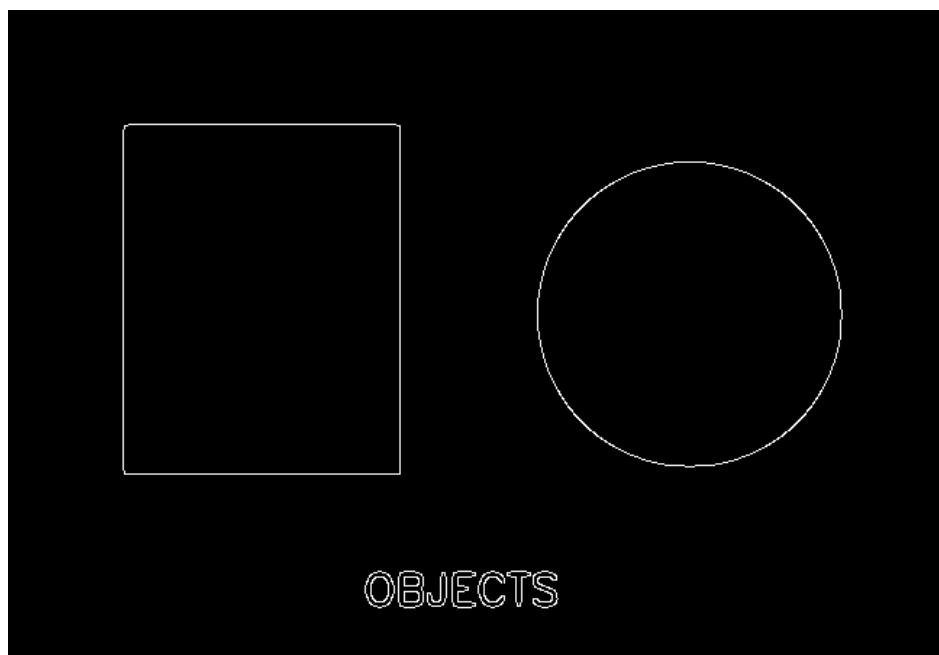
Input Image and Output Images:



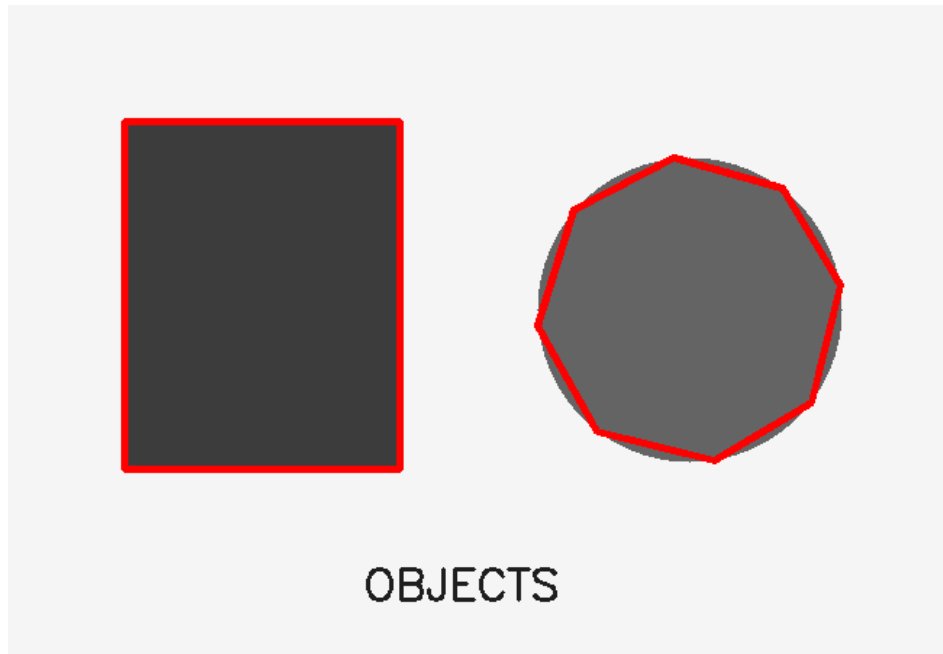
Input image



Grayscale/preprocessed image



Canny edge output



Detected shapes

Result: The integrated workflow successfully detected the main object boundaries in the sample image. The quality depends on preprocessing and Canny threshold values. The method is simple and works well for objects with clear boundaries.

